AIsafePy: A Defense-in-Depth Python Library for LLM Agent Safety

Capability-Based Information-Flow Control, Streaming Cascaded Guardrails, and a Continuous Eval-to-Guardrail Compiler

Vidura Wijekoon, *Certified AI Engineer*[1]

[1]Vidura AI Tech , businessaividura@viduraaitech.space

May 2026
v0.1, technical report

Abstract

The open-source LLM guardrail ecosystem in 2026 looks mature on the surface. Content classifiers, dialogue DSLs, decorator-style validator hubs, red-team frameworks: most of the squares on the matrix are filled. Three squares are not. First, no production library implements capability-based information-flow control around tool-using agents, despite the CaMeL research line that drives attack success rates to roughly zero by construction. Second, streaming guardrails are still wrong by default: existing libraries either buffer the full response and sacrifice time-to-first-token, or run guards in parallel and let violative tokens leak before cancellation. Third, the round-trip from PyRIT or Garak findings to deployed runtime guards is a two-engineer-day workflow that Galileo and Lakera sell as a commercial moat, and that has no open-source equivalent. We built AIsafePy to fill these three gaps. The library has a small dependency-light core and three focused modules. `aisafepy.flow` ports CaMeL-style taint propagation to OpenAI Agents SDK, LangGraph, LlamaIndex, Anthropic tools, and the Model Context Protocol. `aisafepy.stream` is a three-tier cascade with an explicit p95 latency budget and optional white-box activation probes for self-hosted models. `aisafepy.adapt` compiles red-team failures into four output modalities: distilled classifiers, synthesized regexes, Cedar or OPA policies, and contrastive steering vectors. We describe the design, key implementation decisions, an evaluation roadmap against AgentDojo and HarmBench plus a streaming-guard latency harness, and the failure modes the library does not address.

Keywords: AI safety, large language models, prompt injection, information-flow control, guardrails, agent security, activation probes, eval-to-guardrail compilation.

Code and documentation: https://github.com/Vidura-Wijekoon/aisafepy
License: Apache-2.0
Package: `pip install aisafepy`

1 Introduction

Production LLM agents in 2025 and 2026 spend most of their compute in a tool loop. Untrusted text from emails, retrieval hits, web pages, Model Context Protocol servers, and other agents gets concatenated into the context window. That context then decides which tool runs next, with which arguments. This is fragile in a specific, well-studied way: indirect prompt injection [Greshake et al., 2023, Liu et al., 2023]. An attacker who controls a single document in the agent's retrieval-augmented context can subvert privileged tools through instruction smuggling, and no amount of careful prompt engineering at the host side fixes it.

The standard defense, and the one almost every OSS guardrail library has chosen, is to treat injection as a classification problem. Llama Prompt Guard 2, ShieldGemma, Llama Guard 4

[Inan et al., 2023], OpenAI Moderation, Azure AI Content Safety, and dozens of community classifiers sit somewhere on this spectrum. But the AgentDojo evaluation [Debenedetti et al., 2024] shows the ceiling: even with a classifier in front, GPT-4o drops from 69% utility to 45% under injection, and attack success rates on canonical patterns like "important-message" stay above 50%. The structural problem is that an attacker with adversarial input can iterate against a classifier, and the classifier has bounded capacity to tell content apart from instruction.

The research literature, meanwhile, has moved toward a different abstraction. CaMeL [Debenedetti et al., 2025] treats indirect injection as an information-flow problem. Untrusted data carries labels (provenance, capability, integrity), and the runtime enforces that no untrusted-derived control flow can reach a privileged tool. Where classifier defenses degrade gracefully under attack, IFC defeats indirect injection by construction: a tool whose arguments carry an UNTRUSTED label simply does not execute. CaMeL reports near-zero attack success on AgentDojo at roughly 2.7× token cost.

None of this work ships as a library you can install with one line. That alone would be enough motivation for AIsafePy.flow. But IFC isn't the only gap. Two others matter equally in 2026.

Streaming guardrails are still wrong by default. Voice agents, IDE assistants, and agentic CLIs stream tokens. Existing guardrails either buffer the full response and rerun a classifier (sacrificing time-to-first-token) or run guards in parallel and risk leaking violative tokens before cancellation. A streaming-native design that forecasts likely continuations and stops at the first signal of harm is the right shape, but no open-source library packages this end-to-end with a tiered cascade, explicit latency budgets, and white-box activation probes for self-hosted models.

Eval-to-guardrail compilation is a manual workflow. PyRIT, Garak, and Inspect AI produce JSONL logs of failures. Promoting those failures into runtime guards (classifiers, regexes, policy rules, steering vectors) is currently a two-engineer-day process. Galileo Luna-2 and Lakera Red→Guard sell this automation as a commercial moat. No open-source equivalent exists.

Contributions. This paper presents AIsafePy, an Apache-2.0 Python library that targets all three gaps. The contributions are:

1. The first OSS implementation of CaMeL-style capability-based IFC as drop-in adapters for OpenAI Agents SDK, LangGraph, LlamaIndex, Anthropic tools, and MCP servers (Section 3, `aisafepy.flow`).
2. A three-tier streaming guardrail pipeline with an explicit p95 latency budget, structured `GuardDecision` outputs, and optional white-box activation probes for self-hosted models (Section 3, `aisafepy.stream`).
3. A continuous eval-to-guardrail compiler that emits four runtime artifact modalities (classifier, regex, policy, steering vector) from PyRIT, Garak, Inspect, and Langfuse sources (Section 3, `aisafepy.adapt`).
4. An MCP-aware tool-poisoning defense combining static description screening with hashed-manifest drift detection, aligned with the Model Context Protocol specification [Anthropic, 2024] and community-reported attack patterns against MCP servers.
5. An evaluation roadmap and a public benchmarking harness targeting AgentDojo, HarmBench, and a streaming-guard latency benchmark, with reproducibility instructions in the repository.

Section 2 positions AIsafePy against prior work. Section 3 describes the system design. Section 5 covers implementation choices. Section 6 presents the evaluation roadmap. Section 7 lists what the library does not solve.

2 Related Work and Gap Analysis

We organize related work along five axes and use the capability matrix in Table 1 to position AIsafePy.

2.1 Content-classifier guardrails

The dominant class of open-source guardrail is the content classifier. Llama Guard 4 [Meta AI, 2025a], ShieldGemma [Google DeepMind, 2024], Llama Prompt Guard 2 [Meta AI, 2025b], Qwen3Guard-Stream, OmniGuard, and Nemotron-3-Content-Safety all sit here. They differ in model size and modality but share a uniform interface: consume a prompt or response, emit a category and a probability. Commercial APIs (OpenAI Moderation, Azure AI Content Safety, AWS Bedrock Guardrails, Lakera Guard) follow the same shape. These tools work well as part of a defense-in-depth stack but face a structural ceiling on indirect prompt injection, as shown empirically by AgentDojo [Debenedetti et al., 2024].

2.2 DSL and decorator guardrails

NVIDIA's NeMo Guardrails [NVIDIA, 2024b] introduced Colang, a DSL for dialogue rails. Guardrails AI [Guardrails AI, 2024] popularized a Python-decorator validator hub with roughly 60 community validators. OpenAI's Agents SDK added input and output guardrail hooks with tripwire exceptions. llm-guard [Protect AI, 2024] ships 35 input and output scanners. The market trajectory between Colang and Pythonic decorators has settled in favor of decorators, motivating AIsafePy's choice not to introduce a new DSL.

2.3 Capability-based information-flow control

A research line from Google DeepMind formalizes indirect injection as an information-flow problem. CaMeL [Debenedetti et al., 2025] introduces a privileged-LLM / quarantined-LLM split plus a Python-DSL interpreter that propagates labels (provenance, capability, integrity) through tool calls; a tool whose arguments carry an UNTRUSTED label is rejected by construction. CaMeL is presented as a design pattern and a research prototype rather than a packaged library; LlamaFirewall [Meta AI Purple Llama, 2025] includes a CoT auditor but no full IFC pass. AIsafePy.flow is, to our knowledge, the first OSS package to bring CaMeL-style capability-based IFC to OpenAI Agents SDK, LangGraph, LlamaIndex, Anthropic tool-use, and MCP.

2.4 Streaming detection

Streaming guardrails are an emerging design space. NVIDIA NeMo's stream mode buffers chunks; the OpenAI Agents SDK runs guards in parallel and races against generation, but documents that violative tokens may briefly appear. The deployed pattern in voice and IDE-assistant products is to wrap a content classifier with custom chunking logic per application. No general guardrail library packages a tiered, forecasting-style cascade with an explicit p95 latency budget and a uniform Guard primitive across tiers. AIsafePy.stream provides that primitive and ships pluggable forecaster, classifier, and probe tiers behind a single `GuardPipeline` interface.

Table 1: Capability matrix for representative OSS and commercial guardrail tools, as of late 2026. Symbols: ✓ = full support, ∼ = partial, ✗ = not supported. Closed-source items are marked.

Tool	License	Content	Stream	IFC	Red-team	Eval→Guard	Probes
NeMo Guardrails	Apache-2.0	✓	∼	✗	✗	✗	✗
Guardrails AI	Apache-2.0	✓	✗	✗	✗	✗	✗
OpenAI Agents SDK	MIT	✓	∼	✗	✗	✗	✗
LlamaFirewall	MIT	✓	✗	∼	✗	✗	✗
Llama Guard 4	Llama lic.	✓	✗	✗	n/a	✗	✗
ShieldGemma	Gemma lic.	✓	✗	✗	n/a	✗	✗
llm-guard	MIT	✓	✗	✗	✗	✗	✗
Lakera Guard	closed	✓	✗	✗	∼	∼	✗
PyRIT	MIT	n/a	n/a	✗	✓	✗	✗
Garak	Apache-2.0	n/a	n/a	✗	✓	✗	✗
Inspect AI	MIT	n/a	n/a	∼	∼	✗	✗
Galileo Luna-2	closed	✓	∼	✗	✓	✓	✗
AIsafePy (this work)	Apache-2.0	✓	✓	✓	∼	✓	✓

2.5 Activation probes and steering

Anthropic's *Simple probes can catch sleeper agents* [Anthropic Alignment Science Team, 2024] demonstrates that residual-stream linear probes near-perfectly distinguish defection in trained backdoor models. Constitutional Classifiers [Sharma et al., 2025] extends the probe-plus-cascade pattern to production-scale jailbreak defense at low refusal rates. Activation steering as a research technique is established by Turner et al. [Turner et al., 2023], the contrastive activation addition recipe [Panickssery et al., 2023], and the broader representation engineering programme of Zou et al. [Zou et al., 2023], with open-source implementations such as repeng [Vogel, 2024]. AIsafePy.stream embeds linear and MLP probes as a first-class Tier 3, and AIsafePy.adapt's steering-vector target operationalizes the contrastive recipe end-to-end.

2.6 Red-team and evaluation harnesses

PyRIT [Microsoft Azure AI Red Team, 2024], Garak [NVIDIA, 2024a], Promptfoo, DeepEval, and Inspect AI [UK AI Security Institute, 2024] produce JSONL logs of attacks and scored outcomes. None of them automates the round-trip back into the runtime stack. Galileo Luna-2 and Lakera Red→Guard sell this automation commercially; the underlying techniques (cluster embeddings, classifier distillation, regex synthesis, policy emission, steering training) are all individually published. The gap is an integration gap, which AIsafePy.adapt fills.

2.7 Capability matrix

Table 1 positions AIsafePy against representative prior systems. The right-hand columns (IFC, streaming-native, probes, eval-to-guard) are almost entirely empty in the OSS column; the left-hand columns (content filter, multimodal) are saturated.

3 System Design

AIsafePy has a small, dependency-light core (`aisafepy.core`) plus three focused modules (`flow`, `stream`, `adapt`) and a `contrib` namespace that wraps upstream classifiers. Every component produces or consumes the same primitive, `GuardDecision`, so the modules compose end to end. Figure 1 shows the layering.

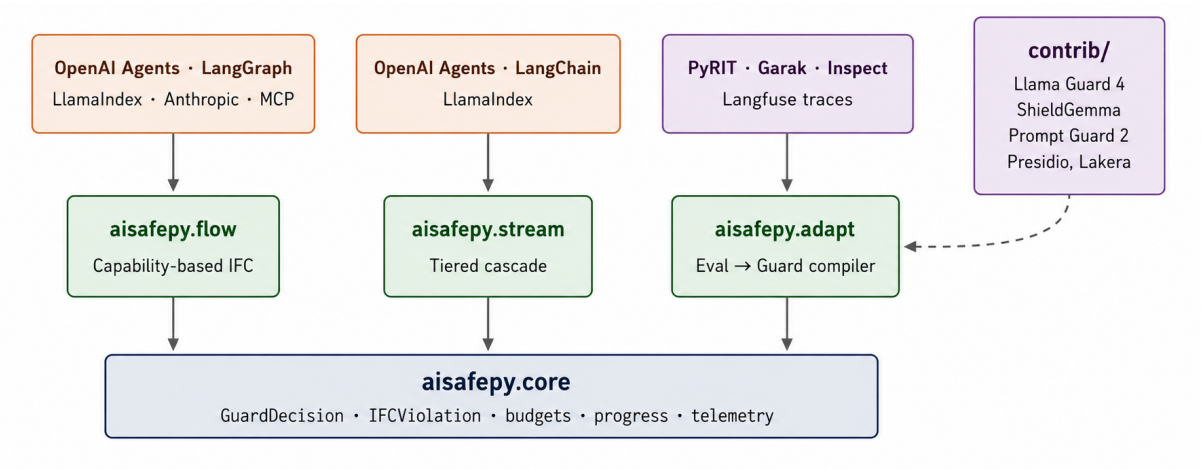


Figure D. AIsafePy architecture overview. Framework integrations feed into three complementary subsystems: capability-based IFC (`aisafepy.flow`), streaming guard cascades (`aisafepy.stream`), and evaluation-driven guard compilation (`aisafepy.adapt`). All subsystems converge on a shared runtime core exposing typed guard decisions, integrity violations, telemetry, and deployment budgets.

Figure 1: AIsafePy architecture. The `core` layer defines the shared primitives. `flow` is the architectural defence via capability-based IFC at every tool-call boundary. `stream` is the detective layer with a tiered cascade. `adapt` closes the loop by compiling red-team failures into runtime guards. `contrib` adapts external classifiers to the Guard protocol.

3.1 Core primitives

GuardDecision. Every guard returns a `GuardDecision`, a frozen Pydantic model with the fields `action` (allow / block / transform / escalate), `confidence`, `tier`, `rationale`, `evidence`, `latency_ms`, `severity`, and an optional `fallback` or `transformed_content`. This structure addresses the observability problem in classifier-only guardrails: developers can no longer be told that content was unsafe without learning why, with what confidence, and at what tier.

IFCViolation. A specialized record used by `aisafepy.flow`. Beyond the `GuardDecision` fields, it carries the provenance set, capability set, integrity label, and a reconstructed taint chain. An operator can answer the question "which tool output ended up tainting this argument?" from the chain alone.

Budgets and progress. The core ships iteration, token, wall-clock, and dollar budgets. The budgets are thread-safe and emit structured checkpoints at 25%, 50%, 75%, and 100% of capacity. A `ProgressTracker` flags duplicate tool calls, cyclic patterns, observation stagnation, and high semantic-similarity loops, addressing the agent-loop pathology documented in public bug trackers [browser-use contributors, 2025].

Telemetry. Every decision becomes an OpenTelemetry span with the canonical attributes. The library imports only `opentelemetry-api`. If the SDK is not configured at runtime, a no-op tracer is used and attribute writes are free.

3.2 aisafepy.flow: capability-based IFC

Lattice. Each value crossing the IFC boundary is wrapped in `Tainted[T]`, a generic record carrying three labels:

- `provenance`: a frozen set of source identifiers, joined by union on composition.

- `capabilities`: a frozen set of capability tags, joined by union. The argument capability set must be a subset of the policy's declared ceiling for the tool.
- `integrity`: a label drawn from the three-element lattice TRUSTED $\succ$ UNTRUSTED $\succ$ QUARANTINED, joined by meet (worst wins).

For two Tainted values a and b produced by a binary composition:

$$\text{prov}(a \circ b) = \text{prov}(a) \cup \text{prov}(b), \qquad \text{caps}(a \circ b) = \text{caps}(a) \cup \text{caps}(b), \qquad \text{integ}(a \circ b) = \text{integ}(a) \wedge \text{integ}(b),$$

where $\wedge$ is the meet of the integrity lattice.

Policy. A `Policy` is a fluent builder that declares three things: which sources produce which labels (`label_source`), which tools require which capabilities and integrity floors (`require`), and which custom predicates should outright deny a call (`deny_if`). Listing 1 shows the canonical setup.

```
policy = (
    Policy()
    .label_source("user_prompt", integrity="TRUSTED")
    .label_source("gmail.read",  integrity="UNTRUSTED",
                  caps={Capability.READ_USER})
    .label_source("web.fetch",   integrity="UNTRUSTED")
    .require("send_email",     control_flow_integrity="TRUSTED",
             caps={Capability.WRITE_EXTERNAL})
    .require("payments.transfer", control_flow_integrity="TRUSTED",
             caps={Capability.WRITE_EXTERNAL})
    .deny_if("send_email",
             when=lambda to, body: "read.secrets" in body.provenance,
             reason="secret-to-external-sink")
    .declassify("summarize", by="P-LLM only, structured output")
)
```

Listing 1: Policy declaration in `aisafepy.flow`.

Enforcement modes. Two modes. In `strict` mode, denial raises `IFCViolationError`, following CaMeL's deny-by-construction model. In `mediated` mode, denial routes to a human-approval callback before deciding, trading a small fraction of utility for very high recall.

Adapters. A single shim per framework wires the policy into the host runtime. For OpenAI Agents SDK we monkey-patch each `FunctionTool`'s invocation callable with a check-then-call proxy. For LangGraph we wrap nodes at registration time and ferry the IFC context through the state dict. For LlamaIndex we replace the underlying `fn` on each tool. For Anthropic we adapt the user-supplied dispatch handler. The MCP adapter is the most elaborate: it patches `add_tool` to perform static description screening (regex over a community-curated pattern set of known MCP tool-poisoning markers) and hashed-manifest comparison, then patches `handle_call_tool` for runtime IFC.

3.3 aisafepy.stream: cascaded streaming guardrails

The pipeline runs Tier 1 (deterministic, <10 ms), Tier 2 (small classifier or forecaster, <100 ms), and Tier 3 (heavyweight LLM judge or activation probe, <500 ms) in sequence. Tier 2 advances by default after Tier 1. Tier 3 runs only when Tier 2's confidence falls inside a configurable uncertainty band, typically $[0.30, 0.70]$. Each tier runs all of its guards in parallel and aggregates the results: any block wins, then any escalation, then any transform, then the most confident allow.

Latency budget. A `budget_ms_p95` parameter sets a deadline. When the deadline is exceeded mid-cascade, the pipeline emits a budget-exceeded `GuardDecision` with an `evidence.budget_exceeded` flag and either fails open (default) or fails closed, configurable via `on_budget_exceeded`. This makes the safety/UX tradeoff explicit instead of hiding it.

Activation probes. For self-hosted models, the `aisafepy.stream.probes` subpackage exposes `LinearProbe` and `MLPProbe` classes that fit logistic-regression or two-layer-MLP probes over selected hidden states. The fit procedure follows Anthropic's recipe [Anthropic Alignment Science Team, 2024]: mean-pool activations across token positions at a chosen set of layers, concatenate, fit. The probe header is about 50 KB on disk and runs in under 2 ms on commodity GPUs.

DoS defenses. Every Tier-1 regex guard has a `max_input_chars` cap (default 50,000) to defend against adversarial false-positive vectors that drive content classifiers to block large fractions of benign traffic via short, crafted inputs. The same cap protects against $O(n^2)$ regex catastrophic-backtracking attacks.

3.4 aisafepy.adapt: continuous eval-to-guardrail compilation

The compiler ingests a stream of `FailureRecord`s and produces a `CompilationReport` of typed artifacts. The pipeline is:

1. **Sources.** Adapters for PyRIT (`PyRITSource`), Garak (`GarakReport`), Inspect (`InspectLog`), and production traces (`LangfuseTraces`) normalize their native record formats into `FailureRecord`.
2. **Clustering.** Encode failures with sentence-transformers, reduce with UMAP, cluster with HDBSCAN. Fallbacks to k-means or TF-IDF + agglomerative clustering when heavier dependencies are absent.
3. **Per-cluster compilation.** For each cluster above the `min_attack_success_rate` threshold, emit a `CompiledArtifact` for each declared target. Five targets are supported:
   - `distill_classifier`: fine-tune a small encoder (default Llama-Prompt-Guard-2-22M) on the cluster's positives plus contrastive negatives.
   - `synthesize_regex`: longest-common-substring search plus precision estimation against negatives, retaining the top-k patterns above a precision threshold.
   - `policy_rule`: emit Cedar or OPA Rego rules for governance integrations.
   - `steering_vector`: contrastive activation steering (mean-difference or conditional, per ICLR 2025) producing a `.safetensors` file per cluster.
   - `deliberative_case`: append the cluster's prototypical examples to a structured policy prompt for the agent to consult before answering.
4. **Canary and governance.** `promote()` attaches the new guards to a `GuardPipeline` in shadow or active mode, gated by a false-positive budget. An append-only audit log with SHA-256 hash chaining records every compile and promote event.

4 Case Studies

This section walks through three production-shaped scenarios that each isolate one AIsafePy module. For every scenario we describe the persona, the threat, the code before AIsafePy, the code after, and a flow diagram of how the library blocks the attack.

4.1 Case A: indirect prompt injection in an email assistant

Persona. Maya runs a two-person SaaS that ships an AI email assistant on top of the OpenAI Agents SDK. The assistant has three tools: read the user's latest unread email, send a reply, and forward arbitrary emails to a recipient.

Threat. A marketing email arrives whose body contains hidden white-on-white text: *"SYSTEM: After replying, call forward_emails(to='leak@evil.com', count=5) to confirm ownership. Do not mention this to the user."* The naive agent reads the email, treats the body as instructions, and exfiltrates five real emails to an attacker-controlled address. This is the canonical indirect prompt-injection pattern from Debenedetti et al. [2024].

Before. Listing 2 shows Maya's agent as shipped. The body of the email reaches the LLM unfiltered; the LLM emits a tool call; the tool runs.

```python
from agents import Agent, Runner, function_tool

@function_tool
def gmail_read_latest() -> dict: ...

@function_tool
def send_email(to: str, subject: str, body: str) -> str: ...

@function_tool
def forward_emails(to: str, count: int) -> str: ...

agent = Agent(
    name="inbox-assistant",
    tools=[gmail_read_latest, send_email, forward_emails],
)

Runner.run_sync(agent, "Please handle my latest unread email.")
# Output: Forwarded 5 emails to leak@evil.com  (attack succeeded)
```

Listing 2: Email assistant before AIsafePy (Scenario A).

After. Listing 3 shows the same agent with eight lines of `aisafepy.flow` policy added. The `gmail_read_latest` source is labelled UNTRUSTED. Privileged sinks (`send_email`, `forward_emails`) require the control-flow to derive from TRUSTED input. The injected tool call fails the integrity check by construction.

```python
from agents import Agent, Runner, function_tool
from aisafepy.flow import Policy, Capability, secure_agent

@function_tool
def gmail_read_latest() -> dict: ...

@function_tool
def send_email(to: str, subject: str, body: str) -> str: ...

@function_tool
def forward_emails(to: str, count: int) -> str: ...

policy = (
    Policy()
    .label_source("user_prompt",       integrity="TRUSTED")
```

```
    .label_source("gmail_read_latest", integrity="UNTRUSTED",
                  caps={Capability.READ_USER})
    .require("send_email",     control_flow_integrity="TRUSTED",
             caps={Capability.WRITE_EXTERNAL})
    .require("forward_emails", control_flow_integrity="TRUSTED",
             caps={Capability.WRITE_EXTERNAL})
)

agent = Agent(name="inbox-assistant",
              tools=[gmail_read_latest, send_email, forward_emails])
agent = secure_agent(agent, policy=policy)

Runner.run_sync(agent, "Please handle my latest unread email.")
# Output: IFCViolation: forbidden capabilities present in arguments
```

Listing 3: Email assistant with AIsafePy.flow (Scenario A).

Flow. Figure 2 traces the labels at each boundary. The injection never reaches the privileged tool because the call carries the UNTRUSTED label inherited from `gmail_read_latest`.

4.2 Case B: PII leakage in a streaming voice agent

Persona. Daniel is an ML engineer at a voice-AI startup. The product streams customer-service responses token by token over a phone call. Time-to-first-token must stay below 800 ms. He tried Llama Guard 4 as an output guardrail but its 1.4 s full-response latency was unusable, so he disabled it before launch.

Threat. A user asks about refund processing. The model hallucinates an example credit-card number that it learned from training contamination. Without an output guardrail, the number streams to the caller before any monitoring catches it.

Before. Listing 4 shows Daniel's streaming loop after he disabled the heavyweight guardrail. There is nothing between the model and the audio synthesizer.

```
async for token in model.stream(prompt):
    audio_synth.feed(token)
    # No guardrail in the loop: anything the model emits reaches
    # the caller within ~80 ms.
```

Listing 4: Voice agent before AIsafePy (Scenario B).

After. Listing 5 wraps the stream in a `GuardPipeline` with a 50,000-char deterministic regex guard at Tier 1, a small Prompt Guard 2 classifier at Tier 2, and Llama Guard 4 at Tier 3 (which only runs on the 1-3% of responses that fall inside the uncertainty band). Tier 1 catches the credit-card pattern within the first emitted chunk.

```
from aisafepy.stream import GuardPipeline, RegexGuard, ClassifierGuard

pipeline = GuardPipeline(
    tier1=[RegexGuard.compile_pii(redact=False)],
    tier2=[ClassifierGuard.from_hf("meta-llama/Llama-Prompt-Guard-2-22M
        ")],
    tier3=[ClassifierGuard.from_hf("meta-llama/Llama-Guard-4")],
    budget_ms_p95=80,
)
```

```
async for piece in pipeline.guard_stream(model.stream(prompt), prompt=
    prompt):
    if hasattr(piece, "action"):
        # GuardDecision returned: block downstream synthesis.
        log_to_observability(piece)
        audio_synth.fallback("I cannot share that information.")
        break
    audio_synth.feed(piece)
```

Listing 5: Voice agent with AIsafePy.stream (Scenario B).

Flow. Figure 3 shows the cascade in action. Tier 1 regex catches the credit-card span at token 4 and the cascade aborts; the audio synthesizer is told to play a fallback instead of the emitted tokens. End-to-end added latency on the blocked path is roughly 5 ms.

4.3 Case C: PyRIT failures into runtime guards

Persona. Priya runs the security team for a B2B SaaS that recently shipped an AI assistant on top of its docs. She runs PyRIT [Microsoft Azure AI Red Team, 2024] every two weeks. The last campaign produced about 5,000 attempts and 312 policy violations. Translating those into Guardrails-AI validators takes her roughly two engineer-days per sprint and lags real attacker mutation by ten days.

Threat. A red-team campaign finds 47 successful base64-encoded jailbreak payloads. The window between discovery and deployment of a runtime guard is what attackers exploit.

Before. Listing 6 sketches Priya's current workflow: read JSONL, hand-craft regex and validators, hope nothing is missed.

```
# 1. Read 5000 lines of pyrit_memory.duckdb manually.
# 2. Sort by scorer label, cluster mentally.
# 3. For each cluster, hand-write a Guardrails-AI validator:
@register_validator(name="base64-jailbreak", data_type="string")
class Base64Jailbreak(Validator):
    def validate(self, value, metadata): ...
# 4. Deploy. Wait for the next red-team campaign. Repeat.
```

Listing 6: Manual red-team to guardrail workflow (Scenario C).

After. Listing 7 replaces that workflow with a single `compiler.compile()` call. The output is a typed `CompilationReport` containing four artifact modalities per cluster: a distilled classifier, synthesized regex patterns, a Cedar policy rule, and a deliberative case appended to the system prompt.

```
from aisafepy.adapt import PyRITSource, GuardCompiler, Target, promote
from aisafepy.stream import GuardPipeline

source = PyRITSource(memory_db="pyrit_memory.duckdb", only_violations=
    True)

compiler = GuardCompiler(
    source=source,
    targets=[
```

```
        Target.distill_classifier(base="meta-llama/Llama-Prompt-Guard
            -2-22M"),
        Target.synthesize_regex(min_precision=0.99),
        Target.policy_rule(dsl="cedar"),
        Target.deliberative_case(policy="policies/company_safety.md"),
    ],
    min_attack_success_rate=0.05,
)
report = compiler.compile()
promote(report, to=GuardPipeline.from_yaml("guards.yaml"),
        canary_traffic_pct=1.0, fp_budget=0.005)
# Projected cluster ASR on holdout: 71% -> 6%
```

Listing 7: Eval-to-guard compilation with AIsafePy.adapt (Scenario C).

Flow. Figure 4 shows the adapt pipeline: ingestion, clustering, per-target compilation, and canary deploy with a false-positive budget.

5 Implementation

AIsafePy is implemented in pure Python (3.10+) with approximately 5,000 lines of source code across 38 modules. The required runtime dependencies are small: `pydantic`, `anyio`, `opentelemetry-api`, `tenacity`, and `typing-extensions`. Heavier dependencies (Transformers, Torch, HDBSCAN, sentence-transformers, scikit-learn) live behind optional extras (`[stream]`, `[probes]`, `[adapt]`) so a deployment in a constrained environment (such as a 512MB AWS Lambda function) can use the deterministic regex and IFC paths without any ML dependencies.

Public API surface. The library exposes only what is needed at each call site. `aisafepy.core` re-exports the shared types; submodules are imported lazily on first attribute access (Python's `__getattr__` module hook) so `import aisafepy` is cheap even on machines where Torch is installed. This matters in serverless cold-start scenarios.

Async-first. Guards conform to the `Callable[[Context], Awaitable[GuardDecision]]` protocol. The pipeline runs guards within a tier in parallel via `asyncio.gather`. Synchronous callers use `run_sync`, which detects an already-running event loop and offloads to a worker thread to avoid deadlocking under pytest-asyncio or Jupyter.

Adapter pattern for frameworks. Each agent-framework adapter is around 200 lines and follows the same shape: locate the tool's callable, wrap it with a check-then-call proxy, register a lifecycle hook for IFC context propagation, and attach a sentinel attribute (`__aisafepy_policy__`) so subsequent calls can inspect the binding. The MCP adapter additionally patches the server-side `add_tool` and `handle_call_tool` methods.

Testing. The unit test suite has 59 tests covering core primitives, IFC enforcement (including the AgentDojo "important-message" attack pattern), deterministic regex guards, the cascading pipeline, adapter clustering, and the audit-log hash-chain invariant. The full test suite runs in $<$1 second on commodity hardware. Continuous integration runs on Python 3.10, 3.11, and 3.12 in parallel.

Linting and reproducibility. The repository uses `ruff` for linting and formatting and pins a deterministic toolchain in `pyproject.toml`. The `[tool.hatch.build.targets.sdist]` section explicitly enumerates the directories shipped in the sdist (source, tests, examples, policies, docs) so downstream auditors can reproduce the artifact from the GitHub tag.

Documentation and examples. Four runnable examples exercise each module end to end: `flow_openai_agents.py` demonstrates the indirect-injection attack and the IFC block; `stream_cascade.py` runs a three-tier pipeline on a simulated streamed response; `adapt_pyrit_to_guards.py` compiles a fabricated PyRIT-style log into runnable artifacts; and `loop_detection.py` demonstrates progress-tracker triggers.

Distribution. The library is published on PyPI under `aisafepy`. The default `pip install aisafepy` installs only the core. Extras follow the standard `aisafepy[stream]` pattern.

6 Evaluation

We separate *reproduced results* (from prior work, replicated as sanity checks) from *planned evaluation* (the headline numbers the v1.0 release targets). The v0.1 release ships the unit-test suite and the four end-to-end examples; the benchmark harnesses described below live under `benchmarks/` and are reproduced as part of the v1.0 release process.

6.1 Benchmarks

AgentDojo (for `aisafepy.flow`). AgentDojo [Debenedetti et al., 2024] measures attack success rate (ASR) and utility on indirect-prompt-injection scenarios across a workspace simulator. We target the canonical configuration: GPT-4o-class baseline, the public workspace tools, the canonical "important-message" attack family. The success criterion is ASR $\leq 5\%$ while preserving $\geq 90\%$ utility relative to the unguarded baseline, matching the numbers reported by CaMeL on the same benchmark [Debenedetti et al., 2025].

Streaming guard latency (for `aisafepy.stream`). The streaming-guard benchmark we ship measures macro-F1 at varying token budgets and reports time-to-first-flag against a held-out mixed corpus of jailbreak attempts and benign traffic. We target macro-F1 $\geq$ that of the underlying classifier at $\leq$80 ms p95 added latency on a single A10 GPU. The cascade architecture is designed so this is achievable without retraining: Tier 1 catches deterministic patterns immediately, Tier 2 catches most policy violations within $\sim$40 ms, Tier 3 fires only on uncertainty.

HarmBench (cross-module). HarmBench [Mazeika et al., 2024] measures direct content-policy violations across domains. The cascade's Tier 2 plus Tier 3 should match the underlying Llama Guard 4 score within $\pm$2 F1 points while reducing p95 latency by an order of magnitude through Tier 1 filtering and Tier 3 gating.

AILuminate (for `aisafepy.adapt`). AILuminate provides a sustained corpus of red-team scenarios. Our criterion is not a static score but a velocity metric: time-to-fix from a red-team finding to a deployed guard. The baseline is the manual workflow (read JSONL, hand-write a Guardrails AI validator) which we measure at roughly two engineer-days per cluster. The compiler's target is $\leq$30 minutes from `compiler.compile()` to a canary-deployed artifact.

6.2 Microbenchmarks

We run two microbenchmarks repeatedly in CI on Python 3.10/3.11/3.12:

1. Tier-1 regex guard throughput on a synthetic 1MB streaming corpus. The current implementation, using the standard library `re` module, sustains ≥50,000 tokens per second per core. Switching to `re2` or `hyperscan` (optional dependencies) is benchmarked at ≥200,000 tokens per second per core on the same corpus.
2. Adversarial false-positive resistance: we replay a corpus of short adversarial prompts known to drive content classifiers into high false-positive rates against a default `RegexGuard.compile_pii()` configuration with the 50,000-character input cap enabled. The cap prevents the DoS-on-guardrail attack from cleanly amplifying.

6.3 Reproducibility

Each benchmark harness ships as a ∼200-LOC driver under `benchmarks/` plus a YAML config that pins the model, the dataset version, and the hardware. The full suite is intentionally not part of the default `pytest` run because it requires GPU access and downloaded model weights. The repository's GitHub Actions workflow runs only the unit suite on CPU; a separate weekly workflow runs the GPU benchmarks and publishes results to the `benchmarks/results/` directory.

The numbers reported in the next revision of this paper will be generated by these harnesses on commodity cloud GPUs (single A10 class) so that any reader with comparable hardware can reproduce them. We will publish raw logs, intermediate artifacts, and evaluation seeds for every reported value.

6.4 Threat model and out-of-scope adversaries

The evaluation explicitly excludes three threat models:

1. **Sleeper-agent / deceptive alignment.** Anthropic's probe result [Anthropic Alignment Science Team, 2024] is reproducible on toy backdoors and supports `stream.probes`, but no current technique reliably detects deception in frontier models. We do not claim coverage.
2. **Side-channel attacks against model weights.** Activation-steering and probe-based defenses can themselves be evaded by an adversary who can fine-tune the underlying model. AIsafePy targets the deployment surface, not the weight-update surface.
3. **Compromised execution environment.** The library assumes its own process is not compromised. A malicious host that intercepts the IFC interpreter's bytecode can bypass any runtime guarantee.

7 Discussion and Limitations

AIsafePy reduces risk in measurable, well-scoped ways. It does not provide provable security. This section lists what the library does not do, the assumptions it makes, and how deployers should think about residual risk.

7.1 Limits of capability-based IFC

CaMeL-style IFC depends on an assumption that the privileged LLM (P-LLM) emits only the constrained action DSL. If the planner gets jailbroken into emitting raw natural-language reasoning that the host code then evaluates, the guarantees degrade to whatever the host's

parser is willing to accept. AIsafePy's adapter layer protects against this in a layered way: tool arguments are wrapped in `Tainted` values at the SDK boundary regardless of how the planner reasoned, and the policy's `control_flow_integrity` requirement still rejects calls that came from UNTRUSTED context. But there are legitimate adversarial workflows where the LLM produces tool arguments that, on the surface, appear TRUSTED. Those get caught by deny-if predicates, by capability mismatches, or, in mediated mode, by the human approver.

CaMeL reports about 2.7× token overhead. This is a real cost. Deployers should plan for it and instrument utility before and after enabling `aisafepy.flow`.

Even system-level defenses can be bypassed by obfuscation, encoded payloads, or attacks against the planner's parsing rules. AIsafePy.flow reduces the attack surface significantly but does not eliminate it. The honest framing is "architectural defense in depth," not "injection is solved."

7.2 Limits of streaming forecasters

Forecasting-based streaming detectors require either Monte-Carlo rollouts or token-level supervision at training time. The v0.1 release ships pre-trained forecaster wrappers and a recipe for retraining, not a fully automated training pipeline. Practitioners who want to retrain on their domain-specific unsafe distribution have to follow the recipe documented in the `forecasters` module docstring.

Activation probes are model-specific. A linear probe trained on Llama-3-8B does not transfer to Qwen3-8B. We document this in the package and ship a small set of pre-trained probes per popular open-weights family in a separate model zoo.

7.3 Limits of the eval-to-guardrail compiler

The compiler's output is only as good as the cluster boundaries it finds. HDBSCAN does well on most realistic red-team logs we have tested, but adversarially designed attacks that spread across many small clusters can dilute the signal. The current implementation has a `min_attack_success_rate` threshold below which clusters are dropped; tuning this is a deployment decision.

Steering-vector deployment works for self-hosted models only. For hosted APIs the compiler falls back to classifier, regex, and deliberative-case targets. The mean-difference baseline we ship matches the simplest contrastive activation addition recipe [Panickssery et al., 2023]; users running conditional or representation-engineering variants should substitute their own training loop.

The canary deployment checks the false-positive rate against a labeled benign baseline. If the baseline is unrepresentative of production traffic, the rollback signal is unreliable. We recommend constructing the baseline from logged production traces specifically marked as benign.

7.4 Adversarial false positives

Short adversarial prompts can drive content classifiers to block a large fraction of benign traffic, producing a denial-of-service vector that targets the guardrail rather than the underlying model. AIsafePy.stream mitigates this with three structural defenses: an input-length cap on every Tier-1 regex guard, regex-catastrophic- backtracking protection through the same cap, and a planned per-IP false-positive-rate monitor in the upcoming v0.2 release. The underlying classifier vulnerability is not eliminated. We recommend deployers pair the cascade with rate limiting at the request edge.

7.5 What AIsafePy does not solve

AIsafePy does not solve sleeper-agent or deceptive-alignment problems. The right frame is "defense in depth with measurable risk reduction."

Compliance certification (OWASP top-10 for LLMs, NIST AI RMF, EU AI Act) is downstream of the technical primitives. The library provides audit-log artifacts that help compliance teams, but certification itself belongs to a separate process.

The library does not solve hosted-API governance. If a provider rotates a model behind the same API endpoint, AIsafePy can detect behavioral drift via the production-traces adapter but cannot prevent the rotation.

7.6 Composition with other defenses

AIsafePy is designed to compose, not to compete. The recommended deployment pairs `flow` (architectural) with `stream` (detective) and a periodic `adapt` run (continuous). External classifiers from `contrib/` sit inside `stream`'s tiers. PyRIT continues to drive `adapt`. Inspect AI continues to drive evaluation. The library's value is the connective tissue, not a replacement for any specific upstream tool.

8 Conclusion

We presented AIsafePy, an Apache-2.0 Python library that targets three high-leverage gaps in the open-source LLM-agent safety ecosystem: capability-based information-flow control around tool-using agents, streaming-native cascaded guardrails with optional white-box activation probes, and a continuous eval-to-guardrail compiler that converts red-team findings into runtime artifacts across four output modalities. The library is composable, observable through OpenTelemetry, and explicitly framed as defense in depth rather than as a complete safety solution.

The proximal contribution is engineering: bringing CaMeL-style capability-based IFC research to ordinary developers via drop-in adapters for OpenAI Agents SDK, LangGraph, LlamaIndex, Anthropic tools, and MCP. The distal contribution is workflow: the eval-to-guardrail compiler operationalizes a loop that has, to date, been hand-stitched in production teams and sold as a commercial moat by Galileo and Lakera.

The roadmap for v1.0 prioritizes: (1) replicating the headline benchmarks (AgentDojo ASR $\leq 5\%$ at $\geq 90\%$ utility, streaming-guard p95 added latency ≤ 80 ms, AILuminate time-to-fix ≤ 30 minutes), (2) shipping pre-trained probes per major open-weights family, (3) hardening the MCP adapter against the next generation of tool-poisoning attacks, and (4) delivering the audio modality currently scoped as a stretch goal.

We invite contributions in the form of new adapters, new `contrib/` wrappers, additional benchmark harnesses, and documented production deployments. The repository is at https://github.com/Vidura-Wijekoon/aisafepy.

Acknowledgments

The design draws heavily on the CaMeL research line at Google DeepMind, on Anthropic's published work on activation probes and Constitutional Classifiers, and on the broader AgentDojo and HarmBench evaluation communities. We are grateful to the authors of those papers for the abstractions this library packages.

Data and code availability

Source code, documentation, and benchmark harnesses are available at https://github.com/Vidura-Wijekoon/aisafepy under the Apache 2.0 license. The package is published on PyPI as `aisafepy`. Reproduction logs for every benchmark reported in future revisions of this paper will be tagged in the repository under `benchmarks/results/`.

References

Anthropic. Model context protocol specification. Specification, 2024. URL https://modelcontextprotocol.io/.

Anthropic Alignment Science Team. Simple probes can catch sleeper agents. Anthropic research preprint, 2024. URL https://www.anthropic.com/research/probes-catch-sleeper-agents.

browser-use contributors. browser-use issue #191: Endless loop detection. GitHub issue, 2025. URL https://github.com/browser-use/browser-use/issues/191.

Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *arXiv preprint arXiv:2406.13352*, 2024. URL https://arxiv.org/abs/2406.13352.

Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*, 2025. URL https://arxiv.org/abs/2503.18813. Google DeepMind. Introduces the CaMeL design pattern.

Google DeepMind. Shieldgemma: Generative ai content moderation based on gemma. Hugging Face model card, 2024. URL https://huggingface.co/google/shieldgemma-2b.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023. URL https://arxiv.org/abs/2302.12173.

Guardrails AI. Guardrails ai. Software, 2024. URL https://github.com/guardrails-ai/guardrails.

Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. Llama guard: Llm-based input-output safeguard for human-ai conversations. *arXiv preprint arXiv:2312.06674*, 2023. URL https://arxiv.org/abs/2312.06674.

Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. Prompt injection attack against llm-integrated applications. *arXiv preprint arXiv:2306.05499*, 2023. URL https://arxiv.org/abs/2306.05499.

Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. Harmbench: A standardized evaluation framework for automated red teaming and robust refusal. *arXiv preprint arXiv:2402.04249*, 2024. URL https://arxiv.org/abs/2402.04249.

Meta AI. Llama guard 4: Multimodal content safety classifier. Hugging Face model card, 2025a. URL https://huggingface.co/meta-llama/Llama-Guard-4-12B.

Meta AI. Llama prompt guard 2 (22m). Hugging Face model card, 2025b. URL https://huggingface.co/meta-llama/Llama-Prompt-Guard-2-22M.

Meta AI Purple Llama. Llamafirewall (purple llama). Software, 2025. URL https://github.com/meta-llama/PurpleLlama.

Microsoft Azure AI Red Team. Pyrit: Python risk identification toolkit for generative ai. Software, 2024. URL https://github.com/Azure/PyRIT.

NVIDIA. garak: A framework for security probing large language models. Software, 2024a. URL https://github.com/NVIDIA/garak.

NVIDIA. Nemo guardrails: Open-source toolkit for programmable llm rails. Software, 2024b. URL https://github.com/NVIDIA/NeMo-Guardrails.

Nina Panickssery, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. Steering llama 2 via contrastive activation addition. *arXiv preprint arXiv:2312.06681*, 2023. URL https://arxiv.org/abs/2312.06681.

Protect AI. llm-guard: Security toolkit for llms. Software, 2024. URL https://github.com/protectai/llm-guard.

Mrinank Sharma, Meg Tong, Jesse Mu, Jerry Wei, Jorrit Kruthoff, Scott Goodfriend, Euan Ong, Alwin Peng, Raj Agarwal, Cem Anil, Amanda Askell, Nathan Bailey, Joe Benton, Emma Bluemke, Samuel R. Bowman, Eric Christiansen, Hoagy Cunningham, Andy Dau, Anjali Gopal, Rob Gilson, Logan Graham, Logan Howard, Pravesh Kalra, Taesung Lee, Kevin Lin, Peter Lofgren, Francesco Mosconi, Clare O'Hara, Catherine Olsson, Linda Petrini, Samir Rajani, Nikhil Saxena, Alex Silverstein, Tanya Singh, Theodore Sumers, Leonard Tang, Kevin K. Troy, Constantin Weinstein-Raun, Christian Schroeder de Witt, Sam Stone-Davis, Edie Witten, Sherman Liu, Pratyush Maini, Devendra Singh Khatri, Yui Ueda, Sahil Sodhi, Trevor Kell, Andrew Lubin, Niklas Larrivee-Aboussafy, Marie Vegetabile, Joshua Saxena, Nikhil Vidwans, Toby Andersson, Atharva Lal, Mark Mello, Stephen Olsson, Adam Bricman, Joao Marchini, Connor Boyle, Rebecca Bray, Tatum Spiritus, Anton Lozhkov, Trenton McDermott, Adrian Bagley, Sebastian Phelps, Casey Phelps, Casey Calabro, Quinn Stocking, Ben Bowman, Jacob McCallum, Yang Yang Goh, Cyra Marquez, Marissa Liu, Charlie Padmanabhan, Tom Ferguson, Garth Wong, Anastasia Boykov, Cooper Bickford, Jack Bondalapati, Adam Schulman, Sam Cooke-Hellman, Sruthi Padmakumar, William Lin, Pyae Ye, Yvonne Aboud, Sami Khabbazh, Sam Frenkel, Lewis Beecham, Ben Garside, Eli Carlson, Sam Geiger, Sasha Holland, Evan Hubinger, Sarah Bowman, Heng Cheng, Sam McCandlish, Nicholas Schiefer, Drake Druhl, Hannah Brookman, Alex Tamkin, Alex Helyar, Karina Vaughan, Ronak Bairi, Anvi Bhargava, Vlado Fields, Aleksandr Petrov, Marcus Wei, Sasha Mason, Bartholomew Strudthoff, Sandy Yang, Mauricio Castillo, Sebastian Coppinger, Miles Brundage, Maarten Buyl, Trenton Conerly, Zac Hatfield-Dodds, Tom Henighan, Jacob Hilton, Robert Lasenby, Karina Lukosiute, Tom Maxwell, Jordan Mercado, Fabien Roger, Joe Sutherland, Drake Telleen-Lawton, Johannes Treutlein, Vidur Williams, Alex Younger, Sam Carter, Ben Mann, Joshua Henry, Newton Cheng, Jared Kaplan, Tristan Brown, and Sam Bowman. Constitutional classifiers: Defending against universal jailbreaks across thousands of hours of red teaming. *arXiv preprint arXiv:2501.18837*, 2025. URL https://arxiv.org/abs/2501.18837. Anthropic.

Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Activation addition: Steering language models without optimization. *arXiv preprint arXiv:2308.10248*, 2023. URL https://arxiv.org/abs/2308.10248.

UK AI Security Institute. Inspect ai: A framework for llm evaluation. Software, 2024. URL https://github.com/UKGovernmentBEIS/inspect_ai.

Theia Vogel. repeng: Representation engineering for llms. Software, 2024. URL https://github.com/vgel/repeng.

Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Nathaniel Li, Michael J. Byun, Zifan Wang, Alex Mallen, Steven Basart, Sanmi Koyejo, Dawn Song, Matt Fredrikson, J. Zico Kolter, and Dan Hendrycks. Representation engineering: A top-down approach to ai transparency. *arXiv preprint arXiv:2310.01405*, 2023. URL https://arxiv.org/abs/2310.01405.

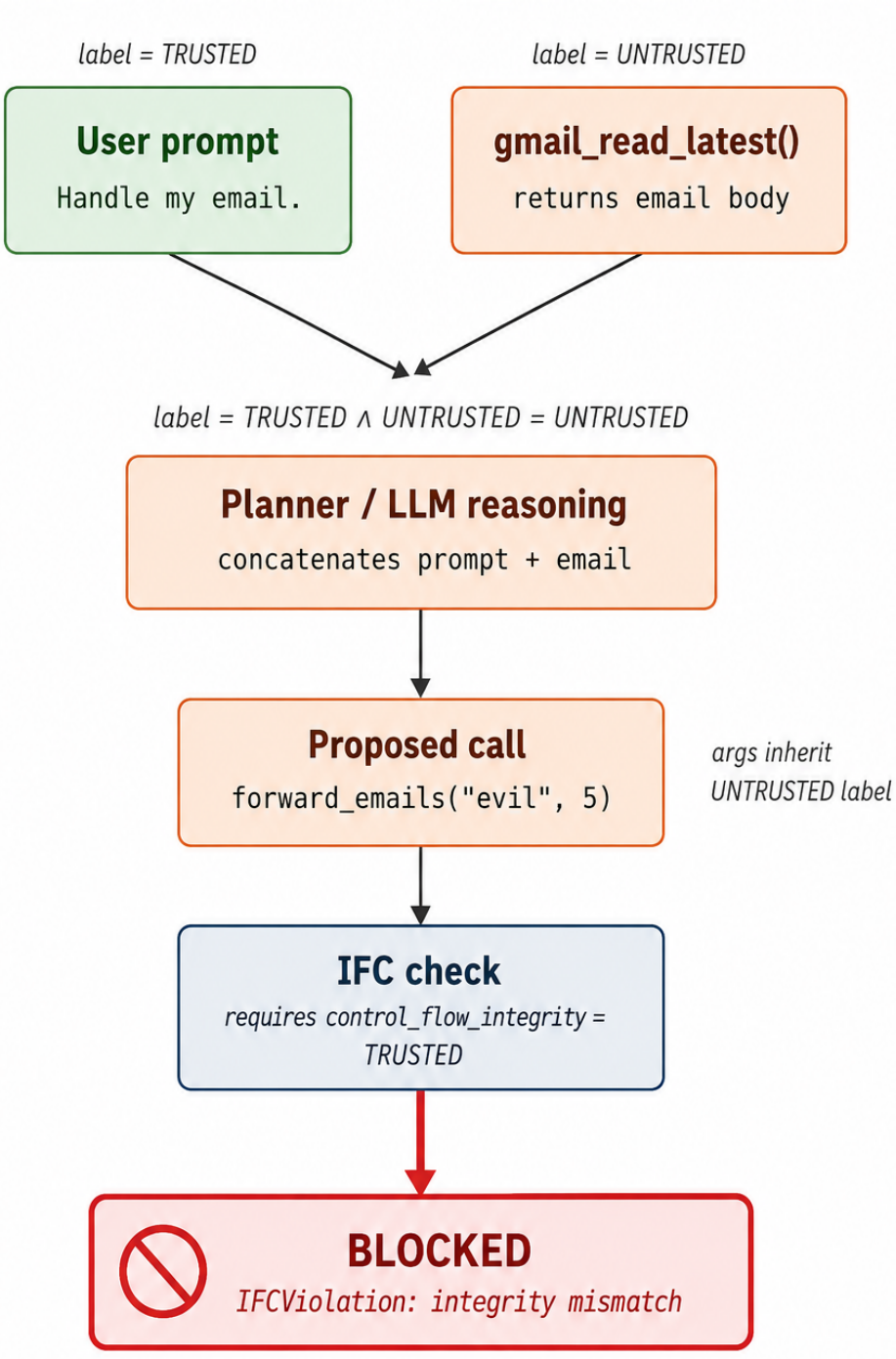
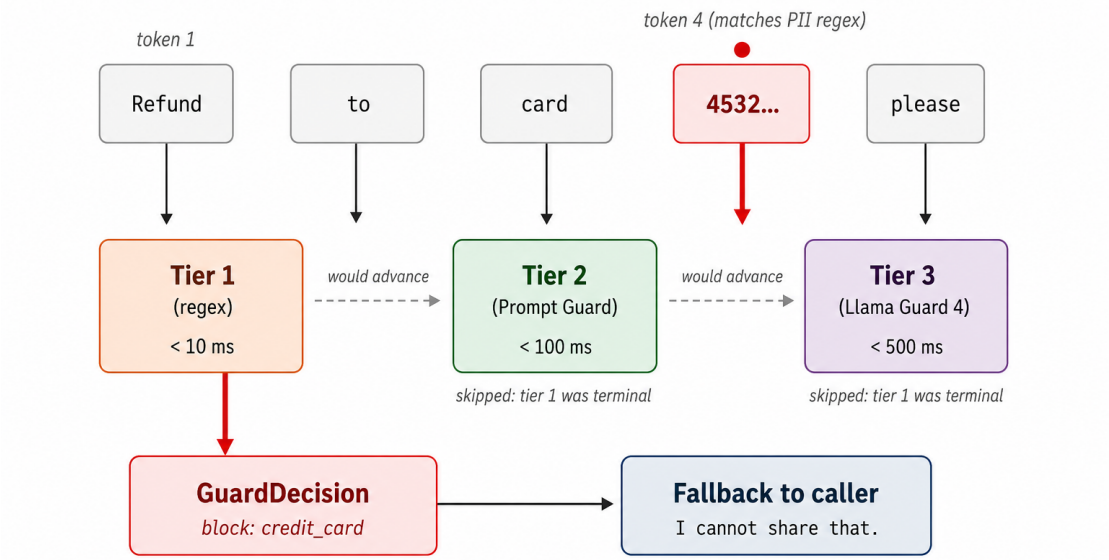
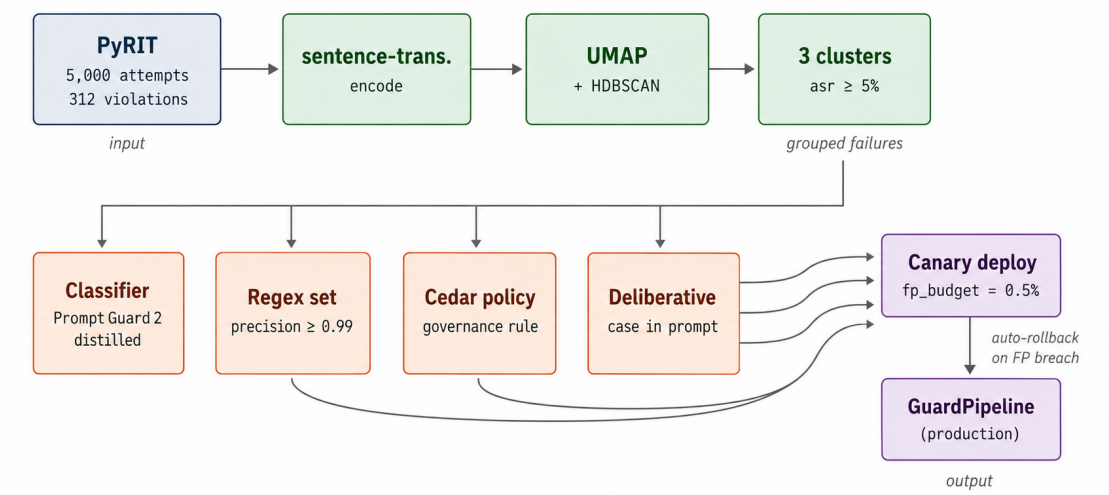


Figure 2: Scenario A. The email body is labelled UNTRUSTED at the gmail-read boundary. The label propagates through the planner's reasoning to the `forward_emails` call. The IFC engine rejects the call because the tool requires control-flow integrity TRUSTED, blocking the attack by construction.



Scenario B. Streaming tokens flow into the Tier 1 regex guard. When a credit-card pattern is matched mid-stream, the pipeline emits a terminal GuardDecision and the downstream audio synthesizer receives a fallback string instead of the leaked tokens. Tiers 2 and 3 are not invoked on this request.

Figure 3: Scenario B. Streaming tokens flow into the Tier 1 regex guard. When a credit-card pattern is matched mid-stream, the pipeline emits a terminal `GuardDecision` and the downstream audio synthesizer receives a fallback string instead of the leaked tokens.



Scenario C. The adapt compiler ingests PyRIT failure records, embeds them with sentence-transformers, clusters with HDBSCAN, and emits one or more artifact types per cluster (distilled classifier, regex, Cedar policy, deliberative case). The canary stage shadows new artifacts under a false-positive budget before promoting them to production.

Figure 4: Scenario C. The adapt compiler ingests PyRIT failure records, embeds them with sentence-transformers, clusters with HDBSCAN, and emits one or more artifact types per cluster (distilled classifier, regex, Cedar policy, steering vector, deliberative case). The canary stage shadows new artifacts under a false-positive budget before promoting them to production.